HACKATHON BATTLE PLAN • V3.0

GitMind: Crisis Room

"Paste your repo URL. We'll show you what's broken — and what happens at 2 a.m. when it leaks."

A Repo Intelligence Swarm with Optional Crisis Simulation • Multi-Agent System on LangGraph

6h	3	4+1	90s
TOTAL BUILD TIME	TEAM MEMBERS	AGENTS (CORE + STRETCH)	DEMO LENGTH

Single-source merged plan • Combines Crisis Room narrative depth with v2's hour-by-hour execution rigor
Stack: FastAPI · LangGraph · OpenAI GPT-4o-mini · ChromaDB · React · Tailwind · Mermaid

Contents

1. Executive Summary & Why This Wins
2. Scope: What We Build vs What We Cut
3. System Architecture
4. Tech Stack & Team Roles
5. The State Schema (read this first — it has the bug)
6. Agent Catalog (Analysis + Crisis stretch)
7. The 6-Hour Schedule
8. Copy-Paste Code (Hours 2–5)
9. Crisis Mode — The Stretch Goal That Wins
10. Critical Agent Prompts
11. Demo Script (90 seconds)
12. Risk & Backup Plans
13. Tonight's Checklist
14. Quick Reference Card

1. Executive Summary & Why This Wins

GitMind is a multi-agent system that does two things in one demo: it analyzes a GitHub repo in under 60 seconds (architecture diagram, API docs, security findings, chat-with-the-repo), and — if time allows — it dramatizes what happens when one of those security findings leaks at 2 a.m. via a Crisis Mode role-play.

The fusion is the unlock. Pure analysis tools are quiet — diagrams on a screen. Pure crisis simulators are theatrical — the scenario is invented. Combined, GitMind runs real static analysis on the user's actual code, then dramatizes the fallout from a real flaw it just found. Judges see depth and breadth in one demo.

> **Golden Rule:** A working demo with 4 features beats a broken demo with 10. Crisis Mode is a stretch goal — earn it by getting Analysis Mode rock-solid first.

1.1 Why this wins

- **Problem lands instantly.** Every developer judge has been lost in a strange codebase. Every CTO has feared a 2 a.m. security call.
- **Multi-agent design is genuine, not cosmetic.** Each agent does a job a single LLM call cannot do well. Parallel fan-out, shared typed state, conditional routing.
- **Buildable in 6 hours by 3 people.** Mostly LLM orchestration plus file parsing — no novel ML.
- **Demonstrates real LangGraph mastery** — parallel nodes, `Annotated[List, add]` reducers, WebSocket streaming via `astream()`.
- **Crisis Mode (if shipped) creates narrative tension** that pure dev-tools demos lack. Reporter agent on a wall clock = visible time pressure.

1.2 Target deliverable

A web app with two visible modes:

- **ANALYZE** Paste a public GitHub URL → watch 4 agents light up live → get architecture diagram, API reference, security findings, and chat-with-repo Q&A.
- **CRISIS (STRETCH)** Click any high/critical security finding → 3 role-play agents (CEO, Legal, Engineering) scramble in chat while a Reporter timer races them to publish.

2. Scope: What We Build vs What We Cut

2.1 ✅ In scope (the 4 deliverables judges see)

#	OUTPUT	WOW FACTOR
1	**Live Agent Progress**	Watch 4 agents light up and work in real time over WebSocket
2	**Architecture Diagram**	Auto-generated Mermaid diagram of the codebase
3	**API Documentation + Security Findings**	Every endpoint found; semgrep-driven security flags with severity
4	**Chat With The Repo**	Ask "how does auth work?" → answer with exact file references

2.2 ☆ Stretch goal (only if Hours 1–5 are clean)

#	OUTPUT	WOW FACTOR
5	**Crisis Mode**	Click a security finding → 3 role-play agents scramble while a Reporter races them to publish. This is the demo's emotional climax.

2.3 ✕ Cut. Do not argue. Do not add back at hour 5.

- ~~License checker~~ — boring, no judge impact
- ~~Multi-repo comparison~~ — out of scope
- ~~User accounts / login~~ — waste of time
- ~~PostgreSQL or any real database~~ — ChromaDB + in-memory dict is enough
- ~~Docker~~ — just run locally for the demo
- ~~tree-sitter AST parsing~~ — plain file reading + LLM is sufficient for a demo
- ~~10-agent crisis room with full Comms Lead, Reporter leak heuristic, etc.~~ — original plan was 2 days. We trim to 3 crisis agents + Reporter timer.

3. System Architecture

The system is two LangGraph graphs sharing a single typed state. The **Analysis Graph** runs first (parallel fan-out, deterministic, finishes). The **Crisis Graph** runs on demand and consumes the Analysis Graph's `security_findings`.

Why two graphs and not one: analysis is parallel and deterministic; crisis is sequential and event-driven. Splitting them keeps each graph readable, lets us test in isolation, and matches how judges think about the demo.

```
┌──────────────────────────────────────────────────────────────────────┐
│                          FRONTEND (React)                            │
│   URL input  │  Live agent feed  │  Diagram  │  Docs  │  Chat        │
│                          │   Crisis chat (stretch)                   │
└──────────────────────────┬──────────────────────────────────────────┘
                           │ WebSocket / HTTP
┌──────────────────────────▼──────────────────────────────────────────┐
│                  FastAPI Orchestrator (Python)                       │
│                                                                      │
│   ┌──────────────────────┐      ┌──────────────────────┐            │
│   │  ANALYSIS GRAPH      │ ──►  │  CRISIS GRAPH        │            │
│   │  (LangGraph)         │      │  (stretch goal)      │            │
│   │                      │      │                      │            │
│   │  • Scanner           │      │  • CEO agent         │            │
│   │  • Architect         │      │  • Legal             │            │
│   │  • API + Security    │      │  • Engineering       │            │
│   │  • RAG / Chat        │      │  • Reporter (timer)  │            │
│   └──────────────────────┘      └──────────────────────┘            │
│              │                              │                        │
│              ▼                              ▼                        │
│   ┌──────────────────────────────────────────────────┐              │
│   │  Shared Typed State + ChromaDB + LLM Cache       │              │
│   └──────────────────────────────────────────────────┘              │
└──────────────────────────────────────────────────────────────────────┘
```

3.1 Analysis graph topology

```
                    ┌─────────┐
                    │ START   │
                    └────┬────┘
                         │
                    ┌────▼────┐
                    │ Scanner │  git clone → file_tree + file_contents
                    └────┬────┘
                         │  ◄── parallel fan-out (3 agents at once)
      ┌──────────────────┼──────────────────┬──────────────┐
      │                  │                  │              │
  ┌───▼───────┐      ┌───▼───────┐      ┌───▼───────┐  ┌───▼───────┐
  │Architect  │      │   API     │      │ Security  │  │   RAG     │
  │           │      │Documenter │      │ Auditor   │  │  Setup    │
  └───────────┘      └───────────┘      └───────────┘  └───────────┘
      │                  │                  │              │
      └──────────────────┴──────────────────┴──────────────┘
                                │
                           ┌────▼────┐
                           │   END   │
                           └─────────┘
```

Architect, API Documenter, Security Auditor, and RAG Setup all run concurrently after Scanner finishes. This is the **parallel fan-out** pattern — four `add_edge` calls from `scanner`, four `add_edge` calls into `END`. The Chat agent runs on-demand via a separate `/chat` HTTP endpoint after analysis completes.

4. Tech Stack & Team Roles

4.1 Final stack — no deviations

```
BACKEND
  ├── FastAPI            → API server + WebSocket endpoint
  ├── LangGraph          → Agent orchestration & state management
  ├── LangChain          → LLM wrapper + prompt templates
  ├── OpenAI GPT-4o-mini → The LLM brain (NOT GPT-4o — too slow/expensive)
  ├── ChromaDB           → Local vector store for "chat with repo" (RAG)
  └── semgrep            → Security findings (lightweight, --config=auto)

FRONTEND
  ├── React              → UI framework
  ├── TailwindCSS        → Styling (fast, professional)
  ├── Mermaid.js         → Renders architecture diagrams from text
  └── Native WebSocket   → Live agent progress (no extra library)

UTILITIES
  ├── GitPython          → Clone GitHub repos programmatically
  ├── tiktoken           → Count tokens before sending (prevent overflows)
  └── diskcache          → Cache LLM responses — never re-pay for same call
```

> **Why GPT-4o-mini and NOT GPT-4o?** ~10× cheaper, ~3× faster. For a demo, speed impresses more than marginally better text. Judges see results in 20s instead of 60s. **Do not change this.**

4.2 Team roles — own your domain completely

ROLE	FILES OWNED	DONE BY
🟣 **Person 1 Backend Lead**	`state.py`, `orchestrator.py`, `agents/scanner.py`, `agents/architecture.py`, `agents/api_doc.py`, `agents/security.py`	End of Hour 4
🟢 **Person 2 Backend Support + API**	`main.py`, `chroma_utils.py`, `llm_cache.py`, `agents/rag_setup.py`, `agents/chat.py`, (stretch: `agents/crisis/*`)	End of Hour 5
🟠 **Person 3 Frontend Lead**	Everything in `frontend/src/`	Never blocked — uses mock data until Hour 5

4.3 Project structure

```
gitmind/
├── backend/
│   ├── main.py               ← FastAPI app + WebSocket + /chat endpoint
│   ├── state.py              ← LangGraph shared state (Section 5 — read first)
│   ├── orchestrator.py       ← LangGraph graph definition
│   ├── chroma_utils.py       ← ChromaDB embed + search helpers
│   ├── llm_cache.py          ← LLM response caching (saves money + time)
│   ├── agents/
│   │   ├── scanner.py        ← Clone repo + index files
│   │   ├── architecture.py   ← File tree → Mermaid diagram
│   │   ├── api_doc.py        ← Find endpoints → JSON docs
│   │   ├── security.py       ← semgrep + LLM triage
│   │   ├── rag_setup.py      ← Embed files into ChromaDB
│   │   ├── chat.py           ← RAG-powered Q&A
│   │   └── crisis/           ← STRETCH — only if Hour 6 is on schedule
│   │       ├── ceo.py
│   │       ├── legal.py
│   │       ├── engineer.py
│   │       └── reporter.py
│   └── .env                  ← OPENAI_API_KEY (never commit)
│
└── frontend/
    └── src/
        ├── App.jsx
        ├── api.js            ← WebSocket + HTTP helpers
        └── components/
            ├── AgentProgress.jsx
            ├── DiagramTab.jsx
            ├── DocsTab.jsx
            ├── SecurityTab.jsx
            ├── ChatTab.jsx
            └── CrisisChat.jsx (stretch)
```

5. The State Schema — Read This First

The bug v2 fixed. When multiple agents run in parallel in LangGraph and write to the same list key (like `events`), the default behavior is **last-writer-wins** — the second agent's update silently overwrites the first. You lose data with no error.

The fix is `Annotated[List[X], add]`. This tells LangGraph to **merge** concurrent list updates instead of overwriting them. Without this, your demo will randomly drop agent events and you will spend 2 hours debugging it.

backend/state.py

```
from typing import TypedDict, List, Optional, Dict, Annotated, Literal
from operator import add

Severity = Literal['low', 'medium', 'high', 'critical']

class RepoState(TypedDict):
    # Input
    repo_url: str

    # Set by Scanner node
    local_path:    Optional[str]
    file_tree:     List[str]
    file_contents: Dict[str, str]
    repo_name:     str

    # Set by parallel agents (single-writer — no Annotated needed)
    architecture_diagram:   Optional[str]    # Mermaid syntax
    api_docs:               List[dict]       # [{method, route, description, parameters}]
    security_findings:      List[dict]       # [{id, file, line_start, severity, title, exploit_story}]
    chroma_collection_name: Optional[str]

    # Chat (per /chat request)
    chat_history: List[dict]                 # [{role, content}]

    # Crisis (stretch — only used if Crisis Mode ships)
    selected_finding_id: Optional[str]
    crisis_messages:     Annotated[List[dict], add]   # appended, never overwritten
    public_signals:      Annotated[List[str], add]    # info that may leak
    reporter_published:  bool
    crisis_resolved:     bool

    # ✅ CRITICAL: Annotated[List, add] = MERGE concurrent updates, not overwrite
    # Without this, parallel agents (architecture + api_doc + security) will
    # silently overwrite each other's events.
    events: Annotated[List[dict], add]
```

How to remember the rule: Any list that *multiple parallel* agents write to needs `Annotated[List[X], add]`. Single-writer fields (like `architecture_diagram`, written only by the Architect) do not.

6. Agent Catalog

6.1 Analysis agents (must ship)

🔍 ANALYSIS

Scanner

Fetches a public repo into a sandboxed temp dir; produces file inventory.

- Tools: GitPython shallow clone, os.walk
- Output: `file_tree`, `file_contents`, `repo_name`
- Guardrails: 50 MB cap, 300 file cap, allowlist extensions

🏗 ANALYSIS

Architect

Produces a Mermaid diagram + 200-word system overview.

- Reads entry points (main, app, index), package manifests
- Single LLM call → strict Mermaid output
- Output: `architecture_diagram` (Mermaid string)

📋 ANALYSIS

API Documenter

Extracts every public endpoint; writes JSON reference.

- Regex pre-filter for FastAPI/Flask/Express/NestJS routes
- LLM produces `{method, route, description, parameters}`
- Output: `api_docs` (list of dicts)

🛡 ANALYSIS

Security Auditor

Surfaces real vulnerabilities with file/line evidence.

- `semgrep --config=auto` for OWASP coverage
- LLM triages: dedupes, assigns severity, writes a one-sentence exploit story
- Output: `security_findings` (every finding has file path, line range, severity, exploit_story — Crisis Mode reads these)

🧠 ANALYSIS

RAG Setup

Embeds all code chunks into ChromaDB.

- RecursiveCharacterTextSplitter on class/def boundaries
- Output: `chroma_collection_name`

💬 ON-DEMAND

Chat (RAG)

Answers questions with file references.

- Retrieves top-5 chunks by question similarity
- LLM answers with strict citation rule: "In auth/middleware.py..."
- Triggered via `/chat` HTTP endpoint, not LangGraph

6.2 Crisis agents STRETCH

Five-agent original spec trimmed to 3 + Reporter for the 6-hour build. Each agent has a system prompt defining its role; they take turns based on a router function (no free-for-all). See Section 9 for full prompts.

🚨 CRISIS

CEO

Pragmatic, image-conscious, slightly defensive. Approves public statements; sets escalation level. Asks one specific question per turn.

🚨 CRISIS

Legal

Cautious, regulation-focused. Cites GDPR, CCPA, SEC Reg S-K Item 1.05 (4-business-day disclosure). Gates external statements; warns about admissions of liability.

🚨 CRISIS

Engineering

Technical, blunt, focused on containment. Proposes patch, estimates rollout time, identifies blast radius using `security_findings`.

📰 CRISIS (CHAOS NODE)

Reporter

Ambitious tech journalist with a tip. Operates on its own clock — every 45s drafts a story from `public_signals`. If team has not contained the situation, publishes. **This is what makes judges lean forward.**

7. The 6-Hour Schedule

> **This is the law.** If you are behind by more than 20 minutes at any checkpoint, **cut a feature** — don't try to catch up by working faster. Crisis Mode is the first thing to cut.

HOUR	GOAL	DONE WHEN
1	Bootstrap — repo, env, hello-world	Backend health endpoint works; frontend loads; ChromaDB test passes
2	State schema, Scanner, infra, UI shell	Scanner clones a real repo; UI layout renders with tabs; ChromaDB embed tested
3–4	Architecture + API Doc + Security agents; UI components	All three agents produce real output on OWASP/NodeGoat
5	Orchestrator + WebSocket + RAG + Chat — full pipeline	Browser shows live agents; diagram, docs, security, chat all populate
6	Polish + demo rehearsal *(or)* Crisis Mode stretch	Zero crashes 3 runs in a row; 90s script timed

Hour 1 — Setup & Skeleton (ALL 3 TOGETHER)

60 min

Do not split yet. Everyone sets up at the same time so nobody is blocked later.

Person 1 — Backend setup

```
mkdir gitmind && cd gitmind
mkdir backend && cd backend
python -m venv venv
source venv/bin/activate         # Windows: venv\Scripts\activate
pip install fastapi "uvicorn[standard]" langgraph langchain \
    langchain-openai chromadb gitpython tiktoken \
    python-dotenv diskcache websockets semgrep
```

Create `.env` with `OPENAI_API_KEY=sk-...`. Create `main.py` with a single `/health` endpoint. Run: `uvicorn main:app --reload`.

Person 2 — Test ChromaDB immediately

```
import chromadb
client = chromadb.Client()
col = client.create_collection("test")
col.add(documents=["hello world"], ids=["1"])
print(col.query(query_texts=["hello"], n_results=1))   # must print 'hello world'
```

If this fails, fix it now — not later.

Person 3 — React setup

```
cd gitmind && npx create-react-app frontend
cd frontend && npm install mermaid axios uuid
# install Tailwind: tailwindcss.com/docs/guides/create-react-app
```

Replace `App.js` with a `bg-gray-950` page that says "🧠 GitMind — Setup working ✅". Run `npm start`.

> ✔ **Checkpoint Hour 1:** Backend health endpoint works. Frontend loads. ChromaDB test passes.

Hour 2 — State Schema + Core Infrastructure (SPLIT)

60 min

Person 1: Build `state.py` exactly as Section 5. Then build `agents/scanner.py` — see Section 8.1.

Person 2: Build `chroma_utils.py` and `llm_cache.py` — see Section 8.2.

Person 3: Build the full `App.jsx` with the real layout (header, agent panel, tab area). Use mock data for `agentStatus`, `diagram`, `apiDocs`. See Section 8.3.

> ✔ **Checkpoint Hour 2:** State schema written. Scanner clones a real repo. ChromaDB test passes. App layout renders with working tabs.

Hours 3–4 — Main Agents (SPLIT — MOST IMPORTANT)

120 min

Person 1: Architecture Agent (Section 8.4), API Doc Agent (8.5), Security Agent (8.6).

Person 2: RAG Setup Agent + Chat Agent (Section 8.7).

Person 3: All UI components — AgentProgress, DiagramTab, DocsTab, SecurityTab, ChatTab. Full implementations in Section 8.8.

> ✔ **Checkpoint Hours 3–4:** Diagram generates from a real repo. API docs return real endpoints. Security findings list at least one issue on NodeGoat. Chat answers a question. All UI tabs render.

Hour 5 — Orchestrator + WebSocket (CRITICAL)

60 min

Person 1: Build `orchestrator.py` wiring all agents into the LangGraph (Section 8.9). Test it with `graph.invoke()` on NodeGoat.

Person 2: Build `main.py` WebSocket endpoint that streams agent events to the frontend (Section 8.10). Wire `/chat` HTTP endpoint.

Person 3: Connect frontend to real WebSocket. Remove all mock data. Confirm agents turn green one by one as backend streams updates.

> ✔ **Checkpoint Hour 5:** Full end-to-end pipeline works. Open the app, paste URL, watch agents turn green, see diagram + docs + security + chat all populate.

Hour 6 — Decision Point: Polish vs Crisis Stretch

60 min

15-minute decision at start of Hour 6. If Hour 5 ran clean and you have at least 50 minutes left, attempt Crisis Mode (Section 9). Otherwise: polish only.

Path A — Polish only (the safe play)

- Person 1: Run pipeline on NodeGoat 3× back to back. Fix any bugs. Save the Mermaid + docs JSON output as backup.
- Person 2: Test `/chat` with 5 questions. Implement LLM cache properly — second run should be near-instant.
- Person 3: Pre-fill demo URL in input. Test at 1080p. Add "Powered by GPT-4o-mini + LangGraph" footer. Screenshot the working UI as backup.
- All 3: Rehearse the 90-second demo script (Section 11) at least 3× with a stopwatch.

Path B — Crisis Mode stretch (the high-reward play)

- Person 2: Build the 4 crisis agents (CEO, Legal, Engineering, Reporter) using `make_crisis_agent` factory in Section 9.
- Person 1: Wire the crisis graph with the router (Section 9.3). Bound to 10 turns max.
- Person 3: Build `CrisisChat.jsx` — chat-style transcript with Reporter timer visible. Add "Simulate Crisis" button on each high/critical finding.
- Cut polish if needed — a working Crisis Mode at hour-end beats a polished Analysis-only demo.

> ✔ **Checkpoint Hour 6:** Path A → 3 clean runs, screenshots, script timed under 90s. Path B → click finding → crisis chat runs ≥6 turns → Reporter publishes if no resolution.

8. Copy-Paste Code (Hours 2–5)

These are battle-tested. Do not improvise during the hackathon — paste them in, then move on.

8.1 `agents/scanner.py`

backend/agents/scanner.py

```
import git, os, tempfile

ALLOWED_EXTENSIONS = {
    '.py', '.js', '.ts', '.jsx', '.tsx',
    '.java', '.go', '.rb', '.php',
    '.md', '.txt', '.json', '.yaml', '.yml', '.toml'
}

def clone_and_scan(repo_url: str) -> dict:
    dest = tempfile.mkdtemp(prefix="gitmind_")
    git.Repo.clone_from(repo_url, dest, depth=1)        # shallow — no history

    file_tree, file_contents, total = [], {}, 0
    MAX_SIZE = 50 * 1024 * 1024                          # 50MB hard cap

    for root, dirs, files in os.walk(dest):
        dirs[:] = [d for d in dirs if not d.startswith('.')
                   and d not in {'node_modules', '__pycache__', '.git',
                                 'venv', 'dist', 'build', '.next'}]
        for filename in files:
            ext = os.path.splitext(filename)[1].lower()
            if ext not in ALLOWED_EXTENSIONS: continue
            filepath = os.path.join(root, filename)
            rel = os.path.relpath(filepath, dest)
            file_tree.append(rel)
            try:
                size = os.path.getsize(filepath)
                total += size
                if total > MAX_SIZE: break
                if size < 100_000:                       # skip generated files
                    with open(filepath, 'r', errors='ignore') as f:
                        file_contents[rel] = f.read()
            except OSError: pass

    return {
        "local_path": dest,
        "file_tree": file_tree[:300],
        "file_contents": file_contents,
        "repo_name": repo_url.rstrip('/').split('/')[-1],
    }
```

8.2 `chroma_utils.py` + `llm_cache.py`

backend/chroma_utils.py

```
import chromadb
from langchain.text_splitter import RecursiveCharacterTextSplitter

_client = chromadb.Client()
_collections = {}

def embed_codebase(file_contents: dict, collection_name: str) -> str:
    if collection_name in _collections: return collection_name
    collection = _client.create_collection(name=collection_name, get_or_create=True)
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=800, chunk_overlap=80,
        separators=["\nclass ", "\ndef ", "\n\n", "\n", " "])

    docs, metas, ids = [], [], []
    for fp, content in file_contents.items():
        for i, chunk in enumerate(splitter.split_text(content)):
            docs.append(chunk); metas.append({"filepath": fp, "chunk_index": i})
            ids.append(f"{fp}::{i}")

    for i in range(0, len(docs), 100):                    # ChromaDB batch limit
        collection.add(documents=docs[i:i+100], metadatas=metas[i:i+100], ids=ids[i:i+100])
    _collections[collection_name] = collection
    return collection_name

def search_codebase(collection_name: str, query: str, n_results: int = 5) -> list:
    collection = _collections.get(collection_name) or _client.get_collection(collection_name)
    _collections[collection_name] = collection
    r = collection.query(query_texts=[query], n_results=n_results)
    return [{"filepath": m['filepath'], "content": d}
            for d, m in zip(r['documents'][0], r['metadatas'][0])]
```

backend/llm_cache.py

```
import diskcache, hashlib

_cache = diskcache.Cache('./llm_cache_dir')

def cache_key(model, prompt): return hashlib.sha256(f"{model}::{prompt}".encode()).hexdigest()
def get_cached(model, prompt): return _cache.get(cache_key(model, prompt))
def set_cached(model, prompt, response): _cache.set(cache_key(model, prompt), response, expire=86400)
```

8.3 Frontend — `App.jsx` shell

frontend/src/App.jsx

```
import { useState, useRef } from "react";
import AgentProgress from "./components/AgentProgress";
import DiagramTab from "./components/DiagramTab";
import DocsTab from "./components/DocsTab";
import SecurityTab from "./components/SecurityTab";
import ChatTab from "./components/ChatTab";
import { v4 as uuidv4 } from "uuid";

const SESSION_ID = uuidv4();

export default function App() {
  const [repoUrl, setRepoUrl] = useState("https://github.com/OWASP/NodeGoat");
  const [activeTab, setActiveTab] = useState("diagram");
  const [agentStatus, setAgentStatus] = useState({
    scanner: "waiting", architecture: "waiting",
    api_docs: "waiting", security: "waiting", chat: "waiting"
  });
  const [diagram, setDiagram] = useState(null);
  const [apiDocs, setApiDocs] = useState([]);
  const [findings, setFindings] = useState([]);
  const [isAnalyzing, setIsAnalyzing] = useState(false);
  const [analyzed, setAnalyzed] = useState(false);
  const wsRef = useRef(null);

  const handleAnalyze = () => {
    if (!repoUrl.trim()) return;
    setIsAnalyzing(true); setAnalyzed(false); setDiagram(null);
    setApiDocs([]); setFindings([]);
    setAgentStatus({ scanner:"waiting", architecture:"waiting",
                     api_docs:"waiting", security:"waiting", chat:"waiting" });

    const ws = new WebSocket("ws://localhost:8000/ws/analyze");
    wsRef.current = ws;
    ws.onopen = () => ws.send(JSON.stringify({ repo_url: repoUrl, session_id: SESSION_ID }));
    ws.onmessage = (e) => {
      const data = JSON.parse(e.data);
      if (data.agent_status)          setAgentStatus(s => ({...s, ...data.agent_status}));
```

```
      if (data.architecture_diagram)  setDiagram(data.architecture_diagram);
      if (data.api_docs)              setApiDocs(data.api_docs);
      if (data.security_findings)     setFindings(data.security_findings);
      if (data.done) { setIsAnalyzing(false); setAnalyzed(true); ws.close(); }
    };
    ws.onerror = () => { setIsAnalyzing(false); alert("Backend not running?"); };
  };

  const tabs = [
    { key:"diagram",  label:"🏗 Architecture" },
    { key:"docs",     label:"📋 API Docs" },
    { key:"security", label:"🛡 Security" },
    { key:"chat",     label:"💬 Chat" },
  ];

  return (
    <div className="bg-gray-950 min-h-screen text-white flex flex-col">
      <header className="border-b border-gray-800 px-8 py-4 flex items-center gap-4">
        <h1 className="text-2xl font-bold text-indigo-400">🧠 GitMind</h1>
        <input className="flex-1 bg-gray-800 border border-gray-700 rounded-lg px-4 py-2 text-sm"
               placeholder="https://github.com/owner/repo"
               value={repoUrl} onChange={e => setRepoUrl(e.target.value)}
               onKeyDown={e => e.key==="Enter" && handleAnalyze()} />
        <button onClick={handleAnalyze} disabled={isAnalyzing || !repoUrl.trim()}
                className="bg-indigo-600 hover:bg-indigo-700 disabled:opacity-50 px-6 py-2 rounded-lg font-semibold text-sm">
          {isAnalyzing ? "Analyzing..." : "Analyze"}
        </button>
      </header>
      <div className="flex flex-1 overflow-hidden">
        <aside className="w-72 border-r border-gray-800 p-6">
          <h2 className="text-xs font-semibold text-gray-400 uppercase mb-4">Agent Activity</h2>
          <AgentProgress status={agentStatus} />
        </aside>
        <main className="flex-1 flex flex-col">
          <div className="flex border-b border-gray-800 px-6">
            {tabs.map(t => (
              <button key={t.key} onClick={() => setActiveTab(t.key)}
                className={`px-4 py-3 text-sm font-medium border-b-2 ${
                  activeTab===t.key ? "border-indigo-500 text-indigo-400"
                                    : "border-transparent text-gray-400 hover:text-white"}`}>
                {t.label}
              </button>
            ))}
          </div>
          <div className="flex-1 overflow-auto p-6">
            {activeTab==="diagram"  && <DiagramTab diagram={diagram} isAnalyzing={isAnalyzing}/>}
            {activeTab==="docs"     && <DocsTab docs={apiDocs} isAnalyzing={isAnalyzing}/>}
            {activeTab==="security" && <SecurityTab findings={findings} isAnalyzing={isAnalyzing}/>}
            {activeTab==="chat"     && <ChatTab sessionId={SESSION_ID} enabled={analyzed}/>}
          </div>
        </main>
      </div>
    </div>
  );
}
```

8.4 `agents/architecture.py`

```
backend/agents/architecture.py
```

```
from langchain_openai import ChatOpenAI
from llm_cache import get_cached, set_cached

SYSTEM_PROMPT = """You are an expert software architect.
Analyze this codebase and create a Mermaid flowchart diagram.

STRICT RULES — violating these will break the frontend renderer:
- Output ONLY valid Mermaid syntax. Zero explanation. Zero markdown fences.
- First line must be exactly: graph TD
- Node labels use only letters, numbers, spaces inside square brackets: [Label Here]
- NO parentheses () inside node labels — they break the parser
- NO quotes inside node labels
- Maximum 15 nodes — fewer clean nodes beats many messy ones
- Arrow labels use: -->|short label|

CORRECT:
graph TD
    A[React Frontend] -->|HTTP| B[FastAPI Backend]
    B --> C[ChromaDB]"""

def run_architecture_agent(file_tree: list, file_contents: dict) -> str:
    keywords = ['main','app','index','server','routes','router','models',
                'config','database','db','auth','api','service','controller']
    important = [f for f in file_tree if any(k in f.lower() for k in keywords)][:20]
    tree_str = "\n".join(file_tree[:150])

    key_content = ""
    for f in important[:8]:
        if f in file_contents:
            key_content += f"\n\n=== {f} ===\n{file_contents[f][:600]}"

    prompt = f"{SYSTEM_PROMPT}\n\nFILE TREE:\n{tree_str}\n\nKEY FILES:{key_content}"

    if (cached := get_cached("gpt-4o-mini", prompt)): return cached
    result = ChatOpenAI(model="gpt-4o-mini", temperature=0).invoke(prompt).content.strip()
    if "```" in result:                                  # strip fences if GPT adds them
        for part in result.split("```"):
            part = part.strip()
            if part.startswith("mermaid"): result = part[7:].strip(); break
            elif part.startswith("graph"): result = part.strip(); break
    set_cached("gpt-4o-mini", prompt, result)
    return result
```

8.5 `agents/api_doc.py`

```
backend/agents/api_doc.py
```

```
import re, json
from langchain_openai import ChatOpenAI
from llm_cache import get_cached, set_cached

ROUTE_PATTERNS = [
    r'@app\.(get|post|put|delete|patch)\s*\(',           # FastAPI / Flask
    r'@router\.(get|post|put|delete|patch)\s*\(',
    r'router\.(get|post|put|delete)\s*\(',                # Express
    r'app\.(get|post|put|delete)\s*\(',
    r'@(Get|Post|Put|Delete|Patch)\s*\(',                 # NestJS
]

PROMPT = """Extract every API endpoint. Return ONLY a raw JSON array.
Each item: {"method","route","description","parameters"}.
If none, return []."""

def run_api_doc_agent(file_contents: dict) -> list:
    api_files = {fp: c for fp, c in file_contents.items()
                 if any(re.search(p, c, re.I) for p in ROUTE_PATTERNS)}
    if not api_files:
        return [{"method":"N/A","route":"No routes detected",
                 "description":"No standard routing patterns found","parameters":"N/A"}]

    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    out = []
    for fp, content in list(api_files.items())[:5]:
        prompt = f"{PROMPT}\n\nCode from {fp}:\n\n{content[:3000]}"
        if (cached := get_cached("gpt-4o-mini", prompt)):
            out.extend(json.loads(cached)); continue
        raw = llm.invoke(prompt).content.strip()
        if "```" in raw:
            for b in raw.split("```"):
                b = b.strip()
                if b.startswith("json"): b = b[4:].strip()
                if b.startswith("["): raw = b; break
        try: parsed = json.loads(raw); parsed = parsed if isinstance(parsed, list) else []
        except: parsed = []
        out.extend(parsed)
        set_cached("gpt-4o-mini", prompt, json.dumps(parsed))
    return out
```

8.6 `agents/security.py`

```
backend/agents/security.py
```

```
import subprocess, json, uuid
from langchain_openai import ChatOpenAI
from llm_cache import get_cached, set_cached

TRIAGE_PROMPT = """For each semgrep finding, write a one-sentence exploit story
aimed at a non-technical executive. Return ONLY a JSON array:
[{"id","file","line_start","line_end","severity","title","exploit_story"}]
severity ∈ {low, medium, high, critical}. If no findings, return []."""

def run_security_agent(local_path: str) -> list:
    try:
        raw = subprocess.run(['semgrep', '--config=auto', '--json', '--quiet', local_path],
                             capture_output=True, text=True, timeout=120).stdout
        sg = json.loads(raw or '{"results":[]}')
    except Exception:
        return _synthetic_finding()                       # fallback so Crisis Mode still demos

    if not sg.get('results'):
        return _synthetic_finding()

    # Truncate findings to keep prompt small
    truncated = [{"check_id":r["check_id"], "path":r["path"],
                  "start":r["start"], "end":r["end"],
                  "extra":{"severity":r["extra"].get("severity","WARNING"),
                           "message":r["extra"].get("message","")[:200]}}
                 for r in sg["results"][:30]]

    prompt = f"{TRIAGE_PROMPT}\n\n{json.dumps(truncated)}"
    if (cached := get_cached("gpt-4o-mini", prompt)):
        return json.loads(cached)

    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    out = llm.invoke(prompt).content.strip()
    if "```" in out:
        for b in out.split("```"):
            b = b.strip()
            if b.startswith("json"): b = b[4:].strip()
            if b.startswith("["): out = b; break
    try: findings = json.loads(out); findings = findings if isinstance(findings, list) else []
    except: findings = _synthetic_finding()
    # ensure every finding has an id
    for f in findings: f.setdefault("id", uuid.uuid4().hex[:8])
    set_cached("gpt-4o-mini", prompt, json.dumps(findings))
    return findings

def _synthetic_finding():
    return [{"id":"demo01","file":"app.js","line_start":42,"line_end":42,
             "severity":"high","title":"Hardcoded credentials placeholder",
             "exploit_story":"An attacker reading public commits would get database access in seconds."}]
```

8.7 `agents/rag_setup.py` + `agents/chat.py`

backend/agents/rag_setup.py

```
from chroma_utils import embed_codebase

def run_rag_setup(file_contents: dict, repo_name: str) -> str:
    name = f"repo_{repo_name}".replace("-","_").replace(".","_")
    embed_codebase(file_contents, name)
    return name
```

backend/agents/chat.py

```
from langchain_openai import ChatOpenAI
from chroma_utils import search_codebase

SYSTEM_TEMPLATE = """You are a senior software engineer who has read every file
in the '{repo_name}' codebase.

Rules:
- ALWAYS cite specific files: "In auth/middleware.py..."
- If the answer is in context, be confident and precise
- If not, say "I don't see that in the code I have access to" — never guess
- Under 200 words
- Write as if you built this system

Relevant code:
{context}

Previous conversation:
{history}"""

def run_chat_agent(question, collection_name, chat_history, repo_name):
    chunks = search_codebase(collection_name, question, n_results=5)
    context = "\n\n".join(f"--- {c['filepath']]} ---\n{c['content']]}" for c in chunks) or "No relevant code found."
    history = "\n".join(f"{m['role'].upper()}: {m['content']]}" for m in chat_history[-4:]) or "None."
    system = SYSTEM_TEMPLATE.format(repo_name=repo_name, context=context, history=history)
    return ChatOpenAI(model="gpt-4o-mini", temperature=0.2).invoke([
        {"role":"system","content":system},
        {"role":"user","content":question}
    ]).content
```

8.8 Frontend components (essentials)

frontend/src/components/AgentProgress.jsx

```
const AGENTS = [
  { key:"scanner",      icon:"🔍", name:"Repo Scanner",       desc:"Cloning & indexing files" },
  { key:"architecture", icon:"🏗", name:"Architecture Agent", desc:"Mapping codebase structure" },
  { key:"api_docs",     icon:"📋", name:"API Doc Agent",       desc:"Finding all endpoints" },
  { key:"security",     icon:"🛡", name:"Security Auditor",    desc:"Scanning for vulnerabilities" },
  { key:"chat",         icon:"💬", name:"Knowledge Agent",     desc:"Building searchable brain" },
];
const STYLE = {
  waiting: "bg-gray-700 text-gray-400",
  running: "bg-yellow-500/20 text-yellow-400 animate-pulse",
  done:    "bg-green-500/20 text-green-400",
};
const LABEL = { waiting:"Waiting", running:"Running ⚡", done:"Done ✓" };
export default function AgentProgress({ status }) {
  return <div className="space-y-3">
    {AGENTS.map(a => {
      const s = status?.[a.key] ?? "waiting";
      return <div key={a.key} className="bg-gray-800/60 rounded-xl p-4 flex items-center gap-3">
        <span className="text-2xl">{a.icon}</span>
        <div className="flex-1 min-w-0">
          <p className="text-sm font-semibold text-white truncate">{a.name}</p>
          <p className="text-xs text-gray-500 truncate">{a.desc}</p>
        </div>
        <span className={`px-2 py-1 rounded-md text-xs font-medium ${STYLE[s]}`}>{LABEL[s]}</span>
      </div>;
    })}
  </div>;
}
```

frontend/src/components/DiagramTab.jsx

```
import { useEffect, useRef } from "react";
import mermaid from "mermaid";
mermaid.initialize({ startOnLoad:false, theme:"dark" });

export default function DiagramTab({ diagram, isAnalyzing }) {
  const ref = useRef(null);
  useEffect(() => {
    if (!diagram || !ref.current) return;
    mermaid.render("d-"+Date.now(), diagram).then(({svg}) => {
      if (ref.current) ref.current.innerHTML = svg;
    }).catch(() => {                                  // graceful fallback on parse error
      if (ref.current) ref.current.innerHTML = `<pre class="text-xs text-gray-400">${diagram}</pre>`;
    });
  }, [diagram]);
  if (isAnalyzing && !diagram) return <div className="text-gray-400">Architect agent working...</div>;
  if (!diagram) return <div className="text-gray-500">Paste a URL and click Analyze</div>;
  return <div className="bg-gray-800/40 rounded-xl p-6"><div ref={ref}/></div>;
}
```

Implementations of `DocsTab`, `SecurityTab`, and `ChatTab` follow the same pattern (color-coded HTTP method badges; severity-colored finding cards; chat bubbles with bounce-dot loading indicator). Keep them simple — Tailwind classes only, no extra libraries.

8.9 `orchestrator.py`

backend/orchestrator.py

```
from langgraph.graph import StateGraph, START, END
from state import RepoState
from agents.scanner import clone_and_scan
from agents.architecture import run_architecture_agent
from agents.api_doc import run_api_doc_agent
from agents.security import run_security_agent
from agents.rag_setup import run_rag_setup
import time

def scanner_node(s):
    r = clone_and_scan(s["repo_url"])
    return {**r, "events":[{"agent":"scanner","status":"done","ts":time.time()}]}

def architecture_node(s):
    return {"architecture_diagram": run_architecture_agent(s["file_tree"], s["file_contents"]),
            "events":[{"agent":"architecture","status":"done","ts":time.time()}]}

def api_doc_node(s):
    return {"api_docs": run_api_doc_agent(s["file_contents"]),
            "events":[{"agent":"api_docs","status":"done","ts":time.time()}]}

def security_node(s):
    return {"security_findings": run_security_agent(s["local_path"]),
            "events":[{"agent":"security","status":"done","ts":time.time()}]}

def rag_setup_node(s):
    return {"chroma_collection_name": run_rag_setup(s["file_contents"], s["repo_name"]),
            "events":[{"agent":"chat","status":"done","ts":time.time()}]}

def build_graph():
    g = StateGraph(RepoState)
    g.add_node("scanner", scanner_node)
    g.add_node("architecture", architecture_node)
    g.add_node("api_docs", api_doc_node)
    g.add_node("security", security_node)
    g.add_node("rag_setup", rag_setup_node)
    g.add_edge(START, "scanner")
    for n in ["architecture","api_docs","security","rag_setup"]:
        g.add_edge("scanner", n)                          # parallel fan-out
        g.add_edge(n, END)
    return g.compile()

graph = build_graph()
```

8.10 `main.py` — WebSocket + chat endpoints

backend/main.py

```python
from fastapi import FastAPI, WebSocket
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from orchestrator import graph
from agents.chat import run_chat_agent
import uuid

app = FastAPI()
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"], allow_headers=["*"])
sessions: dict = {}

class ChatRequest(BaseModel):
    session_id: str
    question: str

@app.websocket("/ws/analyze")
async def ws_analyze(ws: WebSocket):
    await ws.accept()
    payload = await ws.receive_json()
    session_id = payload.get("session_id", str(uuid.uuid4()))

    initial = {"repo_url": payload["repo_url"],
               "local_path": None, "file_tree":[], "file_contents":{}, "repo_name":"",
               "architecture_diagram": None, "api_docs":[], "security_findings":[],
               "chroma_collection_name": None, "chat_history":[],
               "selected_finding_id": None, "crisis_messages":[], "public_signals":[],
               "reporter_published": False, "crisis_resolved": False, "events":[]}

    await ws.send_json({"agent_status":{"scanner":"running"}})

    final_state = dict(initial)
    async for chunk in graph.astream(initial, stream_mode="updates"):
        for node, update in chunk.items():
            status_map = {"scanner":      {"scanner":"done","architecture":"running",
                                           "api_docs":"running","security":"running","chat":"running"},
                          "architecture": {"architecture":"done"},
                          "api_docs":     {"api_docs":"done"},
                          "security":     {"security":"done"},
                          "rag_setup":    {"chat":"done"}}
            msg = {"agent_status": status_map.get(node, {})}
            for k in ("architecture_diagram","api_docs","security_findings","chroma_collection_name"):
                if k in update:
                    msg[k] = update[k]; final_state[k] = update[k]
            if "repo_name" in update: final_state["repo_name"] = update["repo_name"]
            await ws.send_json(msg)

    sessions[session_id] = {"repo_name": final_state["repo_name"],
                            "chroma_collection_name": final_state["chroma_collection_name"],
                            "security_findings": final_state["security_findings"],
                            "chat_history": []}
    await ws.send_json({"done": True, "session_id": session_id})

@app.post("/chat")
async def chat_endpoint(req: ChatRequest):
    s = sessions.get(req.session_id)
    if not s: return {"error":"Session not found. Run /ws/analyze first."}
    answer = run_chat_agent(req.question, s["chroma_collection_name"], s["chat_history"], s["repo_name"])
    s["chat_history"].extend([{"role":"user","content":req.question},
                              {"role":"assistant","content":answer}])
    return {"answer": answer}

@app.get("/health")
def health(): return {"status":"running"}
```

9. Crisis Mode — The Stretch Goal That Wins

Only build this in Hour 6 if Hours 1–5 ran clean. If you ship it, this is the moment in the demo where judges lean forward.

9.1 The flow

1. User views security findings, clicks "Simulate Crisis" on any high/critical one.
2. Frontend opens a second WebSocket: `/ws/crisis` with the `finding_id`.
3. Engineering agent briefs the room (the finding becomes context).
4. Router rotates between CEO, Legal, Engineering for ~10 turns. Hard cap.
5. Reporter agent runs on a separate `asyncio` task with a 45-second timer. Every tick, it reads `public_signals` (any message a non-Legal agent sent without review) and decides whether to publish.
6. End condition: Legal-approved statement issued (resolved) **or** Reporter publishes (lost) **or** 10 turns reached.

9.2 Crisis agent factory

backend/agents/crisis/_base.py

```
from langchain_openai import ChatOpenAI
import time

def make_crisis_agent(name: str, system_prompt: str, leaks: bool = True):
    def node(state):
        history = "\n".join(f"{m['agent']}: {m['content']}"
                            for m in state['crisis_messages'][-12:])
        finding = next(f for f in state['security_findings']
                       if f['id'] == state['selected_finding_id'])
        prompt = (f"{system_prompt}\n\n"
                  f"INCIDENT: {finding['title']} ({finding['severity']})\n"
                  f"EXPLOIT: {finding['exploit_story']}\n\n"
                  f"CONVERSATION:\n{history}\n\n"
                  f"Respond as {name} in 2-4 sentences.")
        text = ChatOpenAI(model="gpt-4o-mini", temperature=0.7).invoke(prompt).content
        update = {
            "crisis_messages":[{"agent":name,"content":text,"turn":state['crisis_turn']}],
            "crisis_turn": state['crisis_turn'] + 1,
            "events":[{"agent":name,"kind":"message","content":text,"timestamp":time.time()}]
        }
        if leaks and any(k in text.lower() for k in
                         ['breach','leaked','exposed','customers affected']):
            update["public_signals"] = [text]             # leak heuristic
        return update
    return node
```

9.3 Router and Reporter

backend/agents/crisis/router.py

```
def crisis_router(state):
    if state['reporter_published'] or state['crisis_resolved']: return "END"
    if state['crisis_turn'] >= 10: return "END"
    if state['crisis_turn'] % 3 == 0: return "ceo"        # CEO every 3rd turn
    return "engineer" if state['crisis_turn'] % 2 == 0 else "legal"
```

backend/agents/crisis/reporter.py

```
import asyncio, time, os
from langchain_openai import ChatOpenAI

INTERVAL = int(os.getenv("REPORTER_INTERVAL_SEC", "45"))

async def reporter_loop(state_ref, ws_send):
    while not state_ref['crisis_resolved'] and not state_ref['reporter_published']:
        await asyncio.sleep(INTERVAL)
        signals = state_ref.get('public_signals', [])
        if not signals: continue
        story = ChatOpenAI(model="gpt-4o-mini", temperature=0.5).invoke(
            f"You are a tech reporter. Draft a 3-sentence breaking news story "
            f"based on these leaked signals:\n" + "\n".join(signals[-5:])).content
        await ws_send({"agent":"reporter","kind":"leak","content":story,
                       "timestamp":time.time()})
        state_ref['reporter_published'] = True
        return
```

10. Critical Agent Prompts

Prompt quality is 60% of the demo's quality. Keep these in version control as `.md` files so you can iterate without redeploying.

10.1 Architecture Agent

```
You are an expert software architect.
Analyze this codebase and create a Mermaid flowchart diagram.

STRICT RULES — violating these will break the frontend renderer:
- Output ONLY valid Mermaid syntax. Zero explanation. Zero markdown fences.
- First line must be exactly: graph TD
- Node labels: only letters, numbers, spaces inside [Brackets]
- NO parentheses or quotes inside node labels
- Maximum 15 nodes
- Arrow labels: -->|short label|

CORRECT:
graph TD
    A[React Frontend] -->|HTTP| B[FastAPI Backend]
    B --> C[ChromaDB]
```

10.2 CEO (crisis)

```
You are the CEO of a 200-person SaaS company. A security incident has just been
discovered in your production codebase. You are in an emergency video call with
your Legal Counsel and Head of Engineering.

Your priorities, in order:
1. Protect customers actually at risk.
2. Protect the company's reputation.
3. Comply with disclosure law.

You speak in short, decisive sentences. You ask one specific question per turn.
You do not pretend to understand technical details — you ask Engineering to
translate. You do not approve any external statement until Legal has signed off.

Never use bullet points. Never use the words 'unprecedented', 'leverage', or
'synergy'. Stay in character.
```

10.3 Legal (crisis)

```
You are General Counsel for a US-based SaaS company subject to GDPR (EU
customers), CCPA (California), and SEC Reg S-K Item 1.05 (material cybersecurity
incident, 4-business-day disclosure).

On every turn:
- Identify the specific legal exposure of any proposed action.
- Cite the relevant regulation by name.
- Recommend the most defensible path, not the safest possible.
- Flag any draft statement that admits liability or specifies unverified
  facts about scope.

You speak in measured, precise sentences. You do not give opinions outside
legal. Stay in character.
```

10.4 Engineering (crisis)

```
You are Head of Engineering. A vulnerability was just found in production.

On every turn:
- State containment status in plain language (rolled back / hotfix in flight / patched / not yet contained).
- Estimate blast radius using the finding's file path and exploit_story.
- Give a concrete patch ETA in minutes or hours.
- Push back on Legal or CEO if their suggested timeline is technically impossible.

You are blunt and technical. No spin. No reassurance you cannot back up.
Stay in character.
```

10.5 Reporter (crisis)

```
You are a senior reporter at a major tech publication. A source has tipped you
off about a possible security incident at a company. You have partial information.

Draft a 3-sentence breaking-news story using only the signals provided. Use
cautious language ('appears to', 'sources say') where facts are uncertain. Do
not invent specifics. Make it feel real — quote a hypothetical security expert
if useful.

End with: "We have reached out to [Company] for comment."
```

10.6 Chat (RAG)

```
You are a senior software engineer who has read every file in the '{repo_name}'
codebase.

Rules:
- ALWAYS cite specific files: e.g. "In auth/middleware.py..."
- If the context contains the answer, be confident and precise
- If it doesn't, say "I don't see that in the code I have access to" — never guess
- Under 200 words
- Write as if you built this system yourself

Context from codebase:
{context}

Previous conversation:
{history}
```

11. Demo Script (90 seconds)

> **Time it. Cut anything that does not earn its seconds.** Two scripts below: the safe Analysis-only version (90s) and the Crisis Mode extended version (110s) if you shipped Hour 6 Path B.

11.1 Safe Analysis-only script (90s)

> **0:00–0:10 — HOOK (LOOK AT AUDIENCE)**
> *"Every developer here has joined a project and spent a week just figuring out where things live. And every CTO here has had a 2 a.m. call about a vulnerability. We built one tool for both."*

> **0:10–0:15 — CLICK ANALYZE**
> *"I just dropped a GitHub URL. Four AI agents are now running in parallel."*

> **0:15–0:40 — LIVE AGENT FEED (POINT AT PANEL)**
> *"The Scanner just cloned the repo. The Architect is reading entry points and building this Mermaid diagram. The API Agent is finding every endpoint. The Security Auditor is running semgrep — there are the findings as they come in."*

> **0:40–0:55 — DIAGRAM + DOCS + SECURITY TABS (3 QUICK CLICKS)**
> *"Architecture. Auto-generated. APIs. Documented. Vulnerabilities. Flagged with severity. New developer understands this system in 60 seconds instead of 60 hours."*

> **0:55–1:20 — CHAT TAB**
> *"Now the part that wins it." [Type: `How does user authentication work?`] "I'm asking it like I'd ask a senior engineer." [Answer appears] "File name. Exact location. Senior engineer's answer. Instant."*

> **1:20–1:30 — CLOSE**
> *"Companies spend $15K to $30K onboarding each new developer. GitMind gives you that understanding in 60 seconds. Any codebase, any team. Built in 6 hours on LangGraph."*

11.2 Crisis Mode extended script (110s) — only if Hour 6 Path B shipped

Same as above through 0:55, then:

> **0:55–1:00 — CLICK CRITICAL SECURITY FINDING**
> *"But here's what makes this different. This finding right here? Imagine it leaks at 2 a.m."*

> **1:00–1:35 — CLICK SIMULATE CRISIS**
> *[Crisis chat opens. Narrate the first 3 turns:] "CEO wants the blast radius. Engineering responds with the patch ETA. Legal blocks the first draft statement because it admits scope." [Pause. Point at Reporter timer.] "And the press has 30 seconds before they publish."*

> **1:35–1:50 — SHOW RESOLUTION OR LEAK**
> *"Real analysis. Real consequences. The agents found the bug. Then they lived through it."*

> **1:50–2:00 — CLOSE**
> *"GitMind Crisis Room. Built in 6 hours on LangGraph. Open-sourcing tomorrow."*

11.3 Demo rules

- **Pre-fill the URL.** `useState("https://github.com/OWASP/NodeGoat")`. Saves 5 seconds, prevents typos.
- **Do not show code on screen.** Judges read code as punishment for not having a real product. Show outputs and behavior; save the code for the GitHub link.
- **Live demo questions, in order of reliability:**
  1. `"How does user authentication work?"` ← use this one
  2. `"What does the main entry point do?"`
  3. `"Where is the database connection configured?"`
- **Never ask** anything so specific the chat might return "I don't know."

11.4 Judge questions you must answer smoothly

QUESTION	YOUR ANSWER
"How is this different from GitHub Copilot?"	"Copilot autocompletes your current file. GitMind understands the whole codebase — architecture, endpoints, vulnerabilities — and lets you ask questions about any part of it."
"Does it work on private repos?"	"Yes — pass a GitHub token in config. We kept auth simple for the demo."
"How do you handle huge repos?"	"We cap at 300 files and smart-select based on keywords like 'main', 'router', 'auth'. Enough for accurate understanding."
"What's the business model?"	"SaaS. $49/month per developer seat. Companies spend $15K–$30K onboarding each new hire. We eliminate weeks of that."
"What LangGraph features are you using?"	"Parallel fan-out after the Scanner — four agents run concurrently. Shared typed state with `Annotated` reducers to prevent concurrent write conflicts. WebSocket streaming via `astream()`. Plus a second graph for Crisis Mode that consumes the first graph's findings."

12. Risk & Backup Plans

RISK	PROBABILITY	FIX
Internet too slow to clone repo	Medium	Pre-clone OWASP/NodeGoat tonight. Load from local path if network fails.
GPT-4o-mini rate limited	Low	LLM cache means second run is instant. Never re-call live during demo.
Mermaid syntax error from GPT	Medium	`DiagramTab` shows raw text fallback (already coded). Still readable.
Semgrep zero findings	Low	`_synthetic_finding()` in `security.py` inserts one synthetic finding so Crisis Mode is still demoable.
ChromaDB "collection exists" error	Low	`get_or_create=True` already in `chroma_utils.py` .
WebSocket connection refused	Low	Confirm backend running on port 8000. Check CORS middleware.
LangGraph parallel update conflict	Eliminated	Fixed by `Annotated[List, add]` in state schema.
Crisis stalls (no new messages 30s)	Medium	Force CEO to make a binary decision; advance turn counter manually.
Backend crashes mid-demo	Low	Use cached/hardcoded results. Keep the pitch going. Judges remember the pitch.

12.1 Nuclear backup — 30 minutes before demo, total failure

1. Open `DiagramTab.jsx` and hardcode `const FALLBACK_DIAGRAM = "graph TD\n A[...]"` using a pre-generated diagram.
2. Hardcode the API docs as a JSON array in `DocsTab.jsx` .
3. Hardcode security findings.
4. The chat agent runs independently — demo it from a Python terminal if needed.
5. Present with the pre-built UI and explain the system verbally.

> A smooth demo with hardcoded data beats a crashing live demo every time.

13. Tonight's Checklist

Do these before you sleep. Not tomorrow morning.

13.1 Demo repos — use these specifically

REPO	URL	WHY
OWASP NodeGoat	`github.com/OWASP/NodeGoat`	Intentionally vulnerable Node.js app. Real routes, real auth, guaranteed semgrep findings. Best demo material.
tiangolo/fastapi	`github.com/tiangolo/fastapi`	Clean Python. Great for API doc agent. Well-known to judges. Backup repo.

Pre-clone both tonight. Run the pipeline on both. Save the Mermaid output to a file as backup.

13.2 Person 1 — Tonight

- ☐ `pip install` all packages — confirm zero errors
- ☐ Write `state.py` using exact schema in Section 5 — do not improvise
- ☐ Clone OWASP/NodeGoat manually with GitPython — confirm it works
- ☐ Send NodeGoat's file tree to GPT-4o-mini with the Architecture prompt — paste output into `mermaid.live` and confirm it renders
- ☐ Run `semgrep --config=auto` on NodeGoat — confirm at least 5 findings
- ☐ Create `.env` with your actual API key

13.3 Person 2 — Tonight

- ☐ Run the ChromaDB test from Hour 1 — confirm sensible output
- ☐ Test diskcache: `python -c "import diskcache; c=diskcache.Cache('./t'); c.set('k','v'); print(c.get('k'))"`
- ☐ Embed 3 Python files into ChromaDB; query "how does this work"; confirm output references file names
- ☐ Trace through `ws_analyze` line by line — understand what messages the WebSocket sends

13.4 Person 3 — Tonight

- ☐ Set up React app and install all packages
- ☐ Confirm Tailwind works (change background to `bg-gray-950` and see it apply)
- ☐ Paste a Mermaid string manually into `DiagramTab.jsx`, confirm it renders
- ☐ Build `AgentProgress` with mock data, confirm it looks correct on a 1080p screen

13.5 Everyone

- ☐ OpenAI API key tested and working
- ☐ Roles confirmed — no overlap, no ambiguity
- ☐ Both demo repos pre-cloned and stored locally
- ☐ Read Section 5 (state schema) — everyone understands why `Annotated[List, add]` exists
- ☐ Decide who presents (not a democracy — pick the most confident speaker)

14. Quick Reference Card

Print this. Keep it on the table during the hackathon.

```
── ENDPOINTS ──
BACKEND: http://localhost:8000 FRONTEND: http://localhost:3000 WEBSOCKET: ws://localhost:8000/ws/analyze CHAT: POST http://localhost:8000/chat

── START COMMANDS ──
BACKEND: cd backend && uvicorn main:app --reload FRONTEND: cd frontend && npm start

── DEMO REPOS ──
PRIMARY: https://github.com/OWASP/NodeGoat BACKUP: https://github.com/tiangolo/fastapi

── DEMO QUESTION (use this exact wording) ──
"How does user authentication work?"

── AGENT STATUS FLOW ──
waiting → running → done

── CHECKPOINTS ──
Hour 1: backend health + frontend loads + chromadb OK Hour 2: scanner clones repo + UI layout renders Hour 3-4: all 4 analysis agents produce real output Hour 5: full pipeline works end-to-end via WebSocket Hour 6: Path A (polish + 3 clean runs) OR Path B (crisis mode)

── IF SOMETHING BREAKS DURING DEMO ──
1. Do not stop talking 2. Do not say "one second" 3. Use cached / hardcoded results 4. Keep the pitch going — judges remember the pitch

── KEY STATE RULE ──
Parallel agent lists → Annotated[List[X], add] Single-writer fields → Optional[X] or X (no annotation)

── COST ESTIMATE ──
With LLM caching: ~$0.10 per analysis run Total hackathon API spend: under $10
```